Introduction to Assembly

Microcomputer Systems

- Components of microcomputer system
- Executing an instruction
- I/O devices
- Programming languages

Components of a Microcomputer System

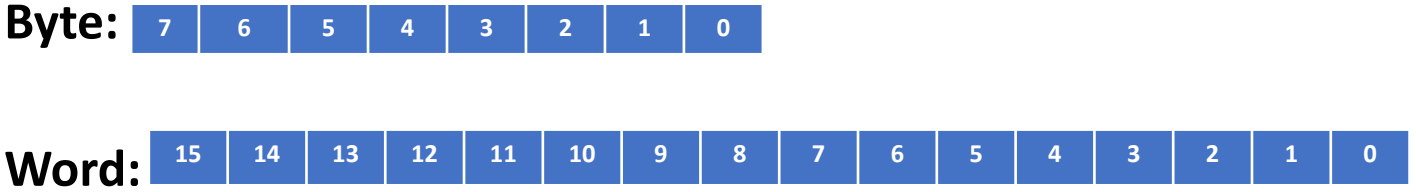
- **Memory**
  - Information processed by the computer is stored in its memory
- **The CPU**
  - In a microcomputer the CPU is a single chip processor known as microprocessor
- **I/O Ports**
  - System board or mother board contains expansion slots which are connectors for additional circuits boards called add in boards. I/O circuits are usually on these boards

Memory

- A memory circuit element can store one bit of data
- Organized into groups that can store eight bits of data
- String of eight bits called a byte
- Each memory byte is identified by address
- The first memory byte has address 0
- The data stored in a memory byte called its contents or values
- The address of a memory byte is fixed and different from any other addresses whereas contents are not fixed
- The contents of memory byte are always eight bits but address depends on the processor. For example some assign 20 bits address whereas some assign 24 bit address

Memory (contd.)

- Bit Position
  - The positions are numbered from right to left starting with 0
  - Low byte comes from memory byte with lower address and high byte comes from memory byte with higher address



- Operations
  - Processor can perform two operations on memory: read(fetch) and write(store)
  - In read processor gets a copy of data and the contents of that location is unchanged whereas in write the data becomes the new content of the location

Memory (contd.)

- RAM and ROM
  - Two kinds of memory circuits: RAM(Random Access Memory) and ROM(Read Only Memory)
  - RAM locations can be read and write but ROM locations only can be read
  - Program instructions and data normally loaded into RAM
  - System programs are stored in ROM
  - RAM memory lost when the power is off but ROM circuits retain their values when the power is off
- Buses
  - Processor communicates with memory and I/O devices by using signals that travel along a set of wires called buses
  - Three kinds of buses: address bus, data bus and control bus

Memory (contd.)

- Suppose a processor uses 20 bits for an address. How many memory bytes can be accessed?
- The number of memory bytes will be $2^{20} = 1{,}048{,}576 = 1\text{MB}$

CPU

- It controls computer by executing programs stored in the memory
- The instructions performed by a CPU is known as instruction set
- Instruction set for each CPU is unique
- 8086 microprocessor has two main components: Execution Unit and Bus Interface Unit
- EU and BIU is connected by an internal bus
- When EU is executing an instruction the BIU fetches up to six bytes of the next instruction and places them in the instruction queue known as instruction prefetch

CPU (contd.)

- **Execution Unit**
  - Used to execute instructions
  - Contains a circuit called Arithmetic and Logic Unit(ALU)
  - The data for the operations are stored in circuits called registers
  - Eight registers for storing data: AX, BX, CX, DX, SI, DI, BP and SP
  - It also contains FLAGS register whose individual bits reflect the result of a computation
- **Bus Interface Unit**
  - Facilitates communication between the EU and the memory or I/O circuits
  - Transmits addresses, data and control signals on the buses
  - Registers are CS, DS, ES, SS and IP holding address of memory locations
  - IP contains the address of the next instruction to be executed by the EU

I/O Ports

- I/O devices are connected to the computer through I/O circuits
- Each of these circuits contains several registers known as I/O ports
- I/O ports have addresses and connected to the bus system
- These addresses are known as I/O addresses and can only be used in input or output instructions
- Data to be input from an I/O device are sent to a port where they can be read by the CPU
- On output CPU writes data to an I/O port
- Two types of I/O ports: Serial and Parallel

I/O Ports (contd.)

- Serial port
  - Transfers one bit at a time
  - Used for slower transfer such as keyboard
- Parallel port
  - Transfers 8 or 16 bits at a time
  - Requires more wiring connections
  - Used for faster data transfer such as disk drives

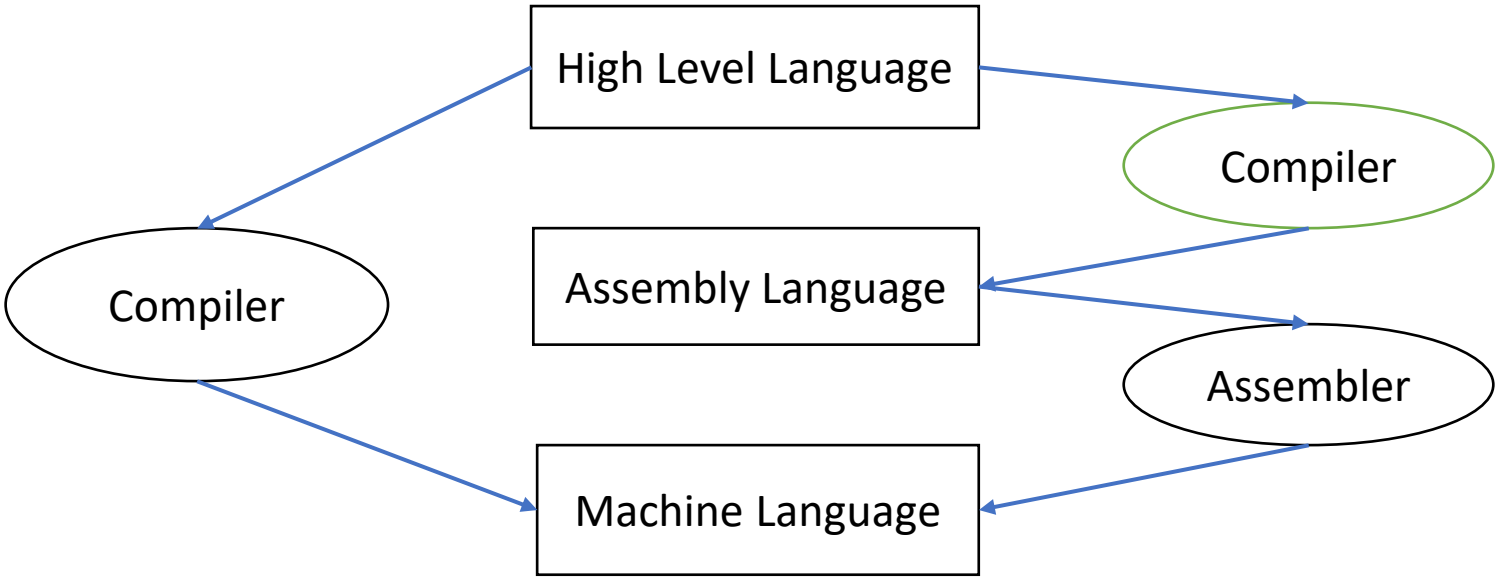
Instruction Execution

- Machine instructions have two parts: Opcode and Operands
- Opcode field which stands for operation code and it specifies the particular operation that is to be performed. Each operation has its unique opcode.
- Operands fields which specify where to get the source and destination operands for the operation specified by the opcode. The source/destination of operands can be a constant, the memory or one of the general-purpose registers.
- The steps of executing an instruction(the fetch-execution cycle) are:
  - Fetch
  1. Fetch an instruction from memory
  2. Decode the instruction to determine the operation
  3. Fetch data from memory if necessary
  - Execution
  1. Perform the operation on the data
  2. Store the result in memory if needed

Programming Languages

- Machine language
  - A CPU can only execute machine language instructions
  - Instructions consist of binary code: 1s and 0s
- Assembly language
  - A programming language that uses symbolic names to represent operations, registers and memory locations.
  - Readability of instructions is better than machine language
  - One-to-one correspondence with machine language instructions to machine code
  - Assemblers translates assembly code to machine code
- High Level Language
  - Compilers translate high-level programs to machine code directly or indirectly via an assembler

Compiler and Assembler



Mapping Between Assembly Language and High Level Language

- Translating High Level Language programs to machine language programs is not a one-to-one mapping
- A High Level Language instruction (usually called a statement) will be translated to one or more machine language instructions

Instruction Class	C	Assembly Language
Data Movement	A=5	MOV A,5

Arithmetic or Logic	B=A+5	MOV AX,A
		ADD AX,5
		MOV B,AX

Data Movement	goto LBL	JMP LBL

Why Learn Assembly Language?

- Accessibility to system hardware
  - Assembly Language is useful for implementing system software
  - Also useful for small embedded system applications
- Space and Time efficiency
  - Understanding sources of program inefficiency
  - Tuning program performance
  - Writing compact code
- Writing assembly programs gives the computer designer the needed deep understanding of the instruction set and how to design one
- To be able to write compilers for High Level Languages, we need to be expert with the machine language. Assembly programming provides this experience.

Number Systems

- Decimal Number System
  - There are ten basic symbols(digits): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
  - The base ten is represented in decimal as 10
  - $3932 = 3 \times 10^3 + 9 \times 10^2 + 3 \times 10^1 + 2 \times 10^0$
- Binary Number System
  - There are two digits: 0 and 1
  - The base 2 is represented in binary as 10
  - $11010 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$
- Hexadecimal Number System
  - There are sixteen digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
  - The base is sixteen is represented in hex by 10
  - $1\mathrm{A} = 1 \times 16^1 + 10 \times 16^0$

Converting Binary and Hex to Decimal

- Hex Number, $8\text{A2Dh} = 8 \times 16^3 + \text{A} \times 16^2 + 2 \times 16^1 + \text{D} \times 16^0$

  $= 8 \times 16^3 + 10 \times 16^2 + 2 \times 16^1 + 13 \times 16^0$

  $= 35373\text{d}$
- Binary Number, $1101\text{b} = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

  $= 13\text{d}$

Converting Decimal to Binary and Hex

- Decimal Number, 11172d

$$11172 = 698 \times 16 + 4$$
$$698 = 43 \times 16 + 10(\text{A})$$
$$43 = 2 \times 16 + 11(\text{B})$$
$$2 = 0 \times 16 + 2$$
$$= 2\text{BA4h}$$

- Decimal Number, 95d

$$95 = 47 \times 2 + 1$$
$$47 = 23 \times 2 + 1$$
$$23 = 11 \times 2 + 1$$
$$11 = 5 \times 2 + 1$$
$$5 = 2 \times 2 + 1$$
$$2 = 1 \times 2 + 0$$
$$1 = 0 \times 2 + 1$$
$$= 1011111\text{b}$$

Conversion Between Hex and Binary

- Hex Number, $2B3Ch = 0010101100111100b$

2	B	3	C
0010	1011	0011	1100

- Binary Number, $1110101010b = 3AAh$

0011	1010	1010
3	A	A

Addition

- Hex Addition

5	B	3	9	h
7	A	F	4	h
D	6	2	D	h

- Binary Addition

1	0	0	1	0	1	1	1	1
			1	1	0	1	1	0
1	0	1	1	0	0	1	0	1

Subtraction

- Hex Subtraction

D	2	6	F	h
B	A	9	4	h
1	7	D	B	h

- Binary Subtraction

1	0	0	1
0	1	1	1
0	0	1	0

One’s Complement Representation

- Representation of 5 and one’s complement of 5

0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0

- If we add 5 and it’s one’s complement we will get 1111111111111111

Two's Complement Representation

- Representation of 5 and two's complement of 5

1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
															1
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1

- If we add 5 and it's two's complement we will get 1000000000000000
- So the two's complement of a number represents its negative form
- Two's complement of two's complement of 5 is 5

Integer Representations

- **Unsigned Integers**
  - Represents a magnitude so it is never negative
  - Largest unsigned integer stored in a byte is 11111111b = FFh =255d
  - Biggest unsigned integer stored in a word is 1111111111111111b = FFFFh = 65535d
  - If least significant bit is 0 then number is even otherwise number is odd
  - Example: Addresses of memory locations, ASCII character codes

- **Signed Integers**
  - It can either be positive or negative
  - Most significant bit is reserved for sign: 0 for positive and 1 for negative
  - Negative integers are stored in a computer as two's complement

Decimal Interpretations

- **Unsigned Decimal Interpretation**
  - Binary to decimal conversion
- **Signed Decimal Interpretation**
  - If MSB is 0 then signed decimal is same as unsigned decimal
  - If MSB is 1 take two's complement and convert it to decimal

Decimal Interpretation

- Most significant bit of a positive signed integer is 0. So the leading hex digit of a positive signed integer is 0-7h. Integers beginning with 8-Fh have 1 in their sign bit so they are negative
- For a word largest positive signed integer is 7FFFh=32767 and smallest negative signed integer is 8000h=-32768
- For a byte largest positive signed integer is 7Fh=127 and smallest negative signed integer is 80h=--128
- For 0000h-7FFFh and 00h-7Fh, signed decimal=unsigned decimal
- For 8000h-FFFFh, signed decimal =unsigned decimal-65536
- For 80h-FFh,signed decimal =unsigned decimal-256

Character Representation

- ASCII Code
  - Most popular encoding scheme for characters
  - Uses seven bits to code each character so there are total of 128 ASCII codes
  - Only 95 ASCII codes from 32-126 are considered to be printable
  - Others are used for communication control purposes
- Keyboard
  - Identifies a key by generating an ASCII code when the key is pressed
  - For IBM-PC each key is assigned an unique number called scan code

Intel 8086 Family

- 8086 and 8088 Microprocessors
  - 8086 is the first 16-bit microprocessor
  - 8086 has 16-bit data bus where 8088 has 8-bit data bus
  - 8086 has faster clock rate than 8088
  - Less expensive to build a original computer around 8088
- 80286 Microprocessor
  - 16-bit microprocessor
  - Faster than 8086
  - Can operate in real address mode and protected virtual address mode
  - In protected mode, can address 16MB of physical memory
  - Can treat external storage as physical memory

Intel 8086 Family

- 80386 Microprocessors
  - 80386 is the first 32-bit microprocessor
  - 8086 has 32-bit data bus
  - High clock rate
  - Execute instructions in fewer clock cycles
  - Can operate in real and protected mode
  - In protected mode can address 4GB of physical memory and 64TB of virtual memory
- 80486 Microprocessor
  - 32-bit microprocessor
  - Fastest and more powerful processor of the family
  - Has numeric processor, cache memory and more advanced design
  - Three times faster than 80386 running at same clock speed

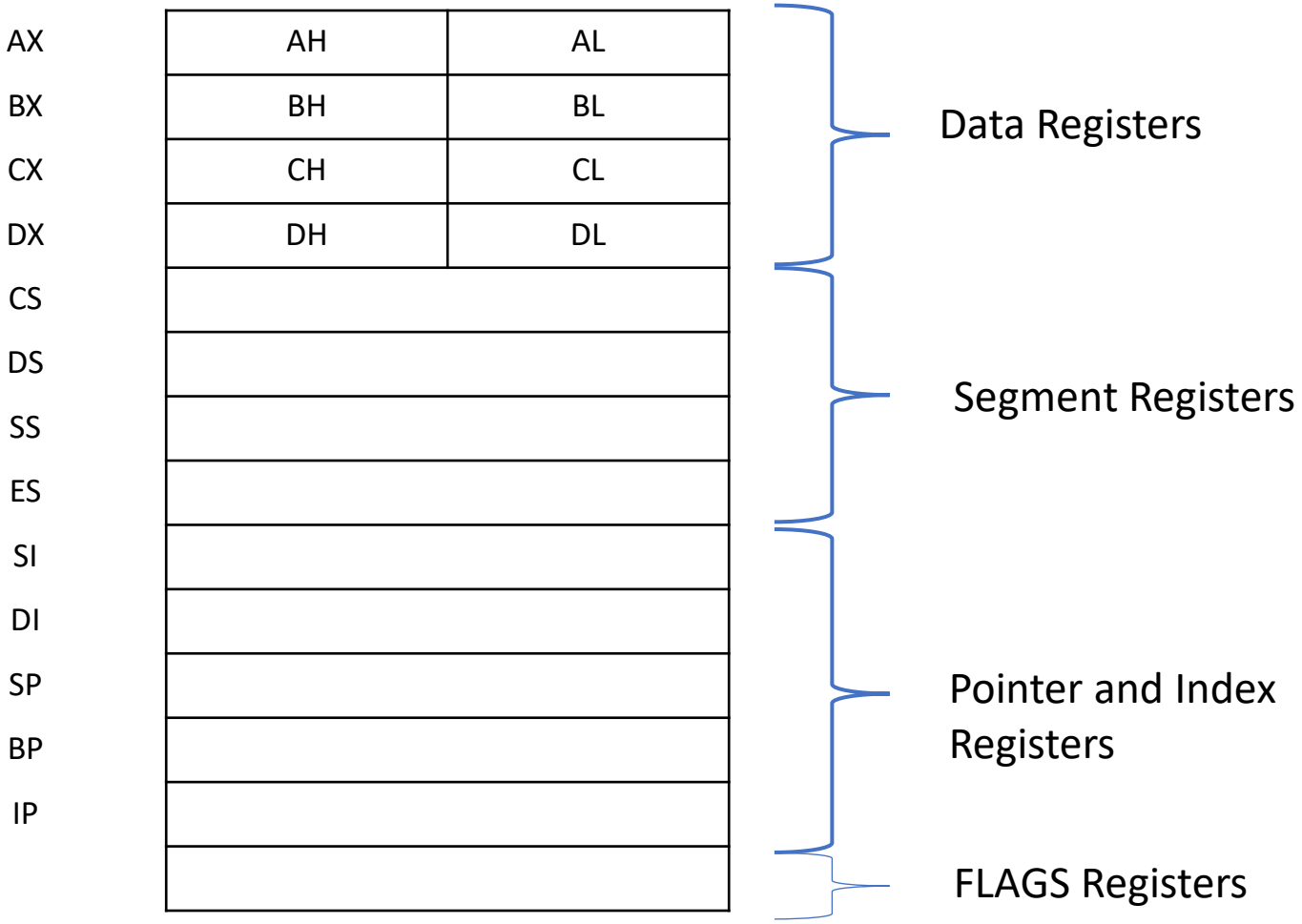
Organization of the 8086/8088 Microprocessors

- Registers
- Data Registers
- Segment Registers
- Pointer and Index Registers
- Instruction Pointer
- FLAGS Register

Registers

- Information inside the microprocessor is stored in registers
- Classified according to the functions they perform
- Data registers hold data for an operation. 8086 has four general data registers
- Address registers hold the address of an instruction or data. Address registers divided into segment, pointer and index registers in 8086
- Status register keeps the current status of the processor. In 8086 status register is called FLAGS register
- Fourteen 16-bit registers in 8086

Registers


AX
BX
CX
DX
CS
DS
SS
ES
SI
DI
SP
BP
IP
AH
AL
BH
BL
CH
CL
DH
DL
Data Registers
Segment Registers
Pointer and Index Registers
FLAGS Registers

Data Registers

- **Accumulator Register(AX)**
  - High byte of AX is called AH and low byte of AX is called AL
  - Used in arithmetic , logic and data transfer instructions
  - Generates shortest machine codes
  - In multiplication and division, one of the numbers involved must be in AX or AL
  - Input and output operations also required AX and AL
- **Base Register(BX)**
  - High byte of BX is called BH and low byte of BX is called BL
  - Serves as an address register

Data Registers

- **Count Register(CX)**
  - High byte of CX is called CH and low byte of CX is called CL
  - CX serves as a loop counter
  - CL is used as a count in instructions that shift and rotate bits
- **Data Register(DX)**
  - High byte of DX is called DH and low byte of DX is called DL
  - Used in multiplication and division
  - Also used in I/O operation

Memory Segment

- 8086 is a 16-bit processor using 20-bit address
- Addresses are too big to fit in a 16 bit register or memory word
- 8086 gets around this problem by partitioning its memory into segments
- Segment Number
  - Memory segment is a block of $2^{16} = 64\text{K}$ consecutive memory bytes identified by a segment number starting with 0
  - Segment number is 16 bit so the highest segment number is FFFFh
- Offset
  - Within a segment a memory location is specified by giving an offset
  - This is the number of bytes from the beginning of segment
  - With 64-bit segment the offset can be given as a 16-bit number
  - The first byte in a segment is offset 0 and the last offset in a segment is FFFFh

Physical Address

- A memory location may be specified by providing a segment number and an offset
- Written in the form segment: offset. This form is known as logical address
- A4FB:4872h means offset 4872h within segment A4FBh
- For obtaining 20 bit physical address, 8086 first shifts the segment address four bits to the left and then adds the offset
- The physical address for A4FB:4872 is A9822h

Segment Registers

- Code Segment(CS)
  - Points at the segment containing the current program.
- Data Segment(DS)
  - Generally points at segment where variables are defined. By default BX, SI and DI registers work with DS segment register
- Stack Segment(SS)
  - Points at the segment containing the stack. BP and SP work with SS segment register
- Extra Segment(ES)
  - Extra segment register, it's up to a coder to define its usage.

Segment Registers

- It is possible to store any data in the segment registers but this is not a good idea.
- Segment registers have a very special purpose, pointing at accessible blocks of memory.
- Segment registers work together with general purpose register to access any memory value.
- For example if we would like to access memory at the physical address 12345h (hexadecimal), we should set the DS =1230h and SI = 0045h.
- The address formed with two registers is called an effective address.
- Although BX can form an effective address, BH and BL cannot.

Pointer and Index Registers

- Stack Pointer(SP)
  - Used with SS for accessing the stack segment
- Base Pointer(BP)
  - Used primarily to access data on the stack
  - Access data in other segments
- Source Index(SI)
  - Used to point to memory locations in the data segment addressed by DS
  - Incrementation of SI give easy access to consecutive memory locations
- Destination index(DI)
  - Same function as SI
  - String operations use DI to access memory locations addressed by ES

Instruction Pointer

- It is used to access instructions
- CS contains the segment number of the next instruction and IP contains the offset
- IP is updated each time an instruction is executed
- An instruction can not contain IP as an operand

FLAGS Register

- Indicates the status of microprocessor
- Status flags reflect the result of an instruction executed by the processor
  - For example if an arithmetic operation produce a zero value as a result then the zero flag(ZF) is set to 1
- Control flags enable or disable certain operations of the processor
  - For example if interrupt flag(IF) is set to zero, inputs from the keyboard are ignored by the processor